

Отчёт по лабораторной работе №14

**Программирование в командном процессоре ОС UNIX.
Расширенное программирование**

Иванова Анастасия Сергеевна

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
2.1	Задание 1. Реализация упрощённого механизма семафоров	5
2.2	Задание 2. Реализация команды map	8
2.3	Задание 3. Генерация случайной последовательности букв	10
3	Контрольные вопросы	12
4	Вывод	14

Список иллюстраций

2.1	Создание файла semafor.sh	5
2.2	Листинг semaphore.sh	6
2.3	Изменение прав доступа	7
2.4	Тестирование семафора терминал 1	7
2.5	Тестирование семафора терминал 2	7
2.6	Создание файла my_man.sh	8
2.7	Листинг my_man.sh	9
2.8	Изменение прав доступа	9
2.9	Тестирование my_man.sh	9
2.10	Создание файла random_letters.sh	10
2.11	Листинг random_letters.sh	11
2.12	Изменение прав доступа	11
2.13	Тестирование random_letters.sh	11

1 Цель работы

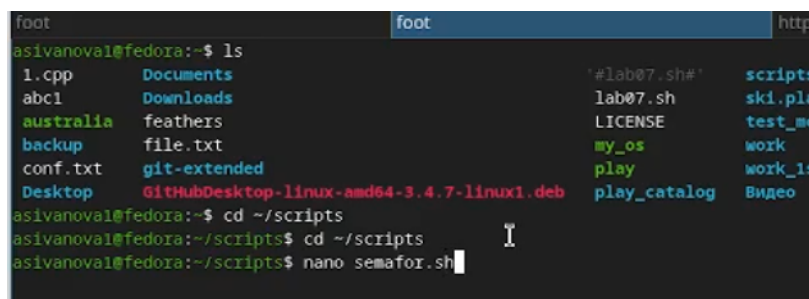
Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Выполнение лабораторной работы

2.1 Задание 1. Реализация упрощённого механизма семафоров

Разработаем скрипт `semafor.sh`, реализующий упрощённый механизм семафоров.

Создание файла `semafor.sh` (рис. 2.1):



The screenshot shows a terminal window with a dark background. The user is logged in as 'asivanova1' on a 'fedora' machine. The terminal shows the following commands and output:

```
asivanova1@fedora:~$ ls
1.cpp      Documents
abc1       Downloads
australia  feathers
backup     file.txt
conf.txt   git-extended
Desktop    GitHubDesktop-linux-amd64-3.4.7-linux1.deb
asivanova1@fedora:~$ cd ~/scripts
asivanova1@fedora:~/scripts$ cd ~/scripts
asivanova1@fedora:~/scripts$ nano semafor.sh
```

Рисунок 2.1: Создание файла `semafor.sh`

Листинг `semafor.sh` (рис. 2.2):

```
#!/bin/bash

LOCK_FILE="/tmp/semaphore.lock"

t1=5
t2=3
```

```

while true; do
    if ln "$LOCK_FILE" "$LOCK_FILE.lock" 2>/dev/null; then
        echo "$$: ресурс захвачен, использую в течение $t2 секунд"
        sleep $t2
        rm "$LOCK_FILE.lock"
        echo "$$: ресурс освобождён"
        break
    else
        echo "$$: ресурс занят, ожидаю $t1 секунд"
        sleep $t1
    fi
done

```

The screenshot shows a terminal window with a dark background. At the top, there are three tabs: 'foot', 'foot', and 'https://mail.ya...'. The active tab is 'foot', and the editor is 'GNU nano 8.5'. The file being edited is 'semaphor.sh'. The script content is as follows:

```

#!/bin/bash

LOCK_FILE="/tmp/semaphore.lock"
t1=5
t2=3

while true; do
    if ln "$LOCK_FILE" "$LOCK_FILE.lock" 2>/dev/null: then
        echo "$$: ресурс захвачен, использую в течение $t2 секунд"
        sleep $t2
        rm "$LOCK_FILE.lock"
        echo "$$: ресурс освобождён"
        break
    else
        echo "$$: ресурс занят, ожидаю $t1 секунд"
        sleep $t1
    fi
done

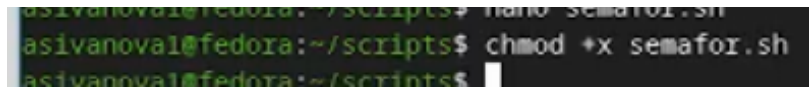
```

The cursor is positioned at the end of the 'done' line.

Рисунок 2.2: Листинг semaphore.sh

Делаем скрипт исполняемым (рис. 2.3):

```
chmod +x semafor.sh
```



```
asivanova1@fedora: ~/scripts$ nano semafor.sh
asivanova1@fedora:~/scripts$ chmod +x semafor.sh
asivanova1@fedora:~/scripts$
```

Рисунок 2.3: Изменение прав доступа

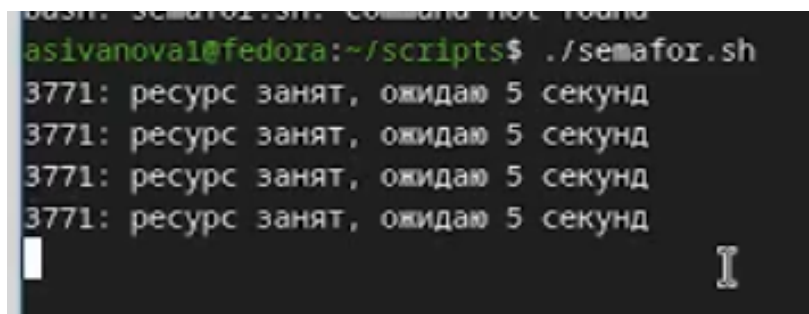
Тестирование работы семафора (рис. 2.4-2.5):

Терминал 1

```
./semafor.sh
```

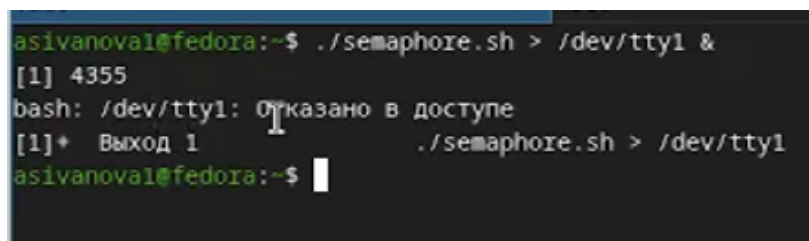
Терминал 2 (фоновый запуск с перенаправлением вывода)

```
./semafor.sh > /dev/tty1 &
```



```
asivanova1@fedora:~/scripts$ ./semafor.sh
3771: ресурс занят, ожидаю 5 секунд
3771: ресурс занят, ожидаю 5 секунд
3771: ресурс занят, ожидаю 5 секунд
3771: ресурс занят, ожидаю 5 секунд
```

Рисунок 2.4: Тестирование семафора терминал 1



```
asivanova1@fedora:~$ ./semafor.sh > /dev/tty1 &
[1] 4355
bash: /dev/tty1: Отказано в доступе
[1]+  Выход 1 ./semafor.sh > /dev/tty1
asivanova1@fedora:~$
```

Рисунок 2.5: Тестирование семафора терминал 2

2.2 Задание 2. Реализация команды man

Разработаем скрипт `my_man.sh`, который выводит справку по указанной команде из каталога `/usr/share/man/man1`.

Создание файла `my_man.sh` (рис. 2.6):

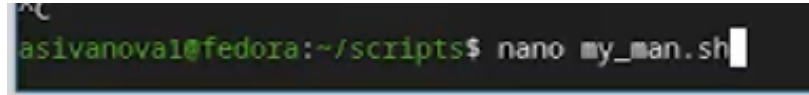


Рисунок 2.6: Создание файла `my_man.sh`

Листинг `my_man.sh` (рис. 2.7):

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

CMD="$1"
MAN_FILE="/usr/share/man/man1/${CMD}.1.gz"

if [ -f "$MAN_FILE" ]; then
    zcat "$MAN_FILE" | less
else
    echo "Справка для команды '$CMD' не найдена"
    exit 1
fi
```

```

GNU nano 8.5 my_man.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

CMD="$1"
MAN_FILE="/usr/share/man/man1/${CMD}.1.gz"

if [ -f "$MAN_FILE" ]; then
    zcat "$MAN_FILE" | less
else
    echo "Справка для команды '$CMD' не найдена"
    exit 1
fi

```

Рисунок 2.7: Листинг my_man.sh

Делаем скрипт исполняемым (рис. 2.8):

```
chmod +x my_man.sh
```

```

asivanova1@fedora:~/scripts$ chmod +x my_man.sh

```

Рисунок 2.8: Изменение прав доступа

Тестирование работы my_man.sh (рис. 2.9):

```
./my_man.sh ls
./my_man.sh unknown_command
```

```

asivanova1@fedora:~/scripts$ ./my_man.sh ls
asivanova1@fedora:~/scripts$ ./my_man.sh unknown_command
Справка для команды 'unknown_command' не найдена
asivanova1@fedora:~/scripts$

```

Рисунок 2.9: Тестирование my_man.sh

2.3 Задание 3. Генерация случайной последовательности букв

Разработаем скрипт `random_letters.sh`, генерирующий случайную последовательность букв латинского алфавита.

Создание файла `random_letters.sh` (рис. 2.10):

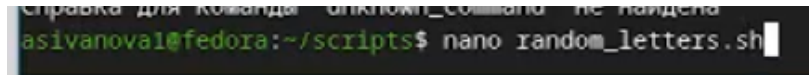


Рисунок 2.10: Создание файла `random_letters.sh`

Листинг `random_letters.sh` (рис. 2.11):

```
#!/bin/bash

LENGTH=${1:-20}

for ((i=0; i<LENGTH; i++)); do
    RAND=$((RANDOM % 26))
    LETTER=$(printf \\$(printf '%03o' $((97 + RAND))))
    echo -n "$LETTER"
done
echo
```

```

GNU nano 8.5 random_letters.sh
#!/bin/bash

LENGTH=${1:-20}

for ((i=0; i<LENGTH; i++)); do
    RAND=$((RANDOM % 26))
    LETTER=$(printf \\$(printf '%03o' $((97 + RAND))))
    echo -n "$LETTER"
done
echo

```

Рисунок 2.11: Листинг random_letters.sh

Делаем скрипт исполняемым (рис. 2.12):

```
chmod +x random_letters.sh
```

```

asivanova@fedora:~/scripts$ nano random_letters.sh
asivanova@fedora:~/scripts$ chmod +x random_letters.sh
asivanova@fedora:~/scripts$

```

Рисунок 2.12: Изменение прав доступа

Тестирование работы random_letters.sh (рис. 2.13):

```
./random_letters.sh
./random_letters.sh 10
```

```

asivanova@fedora:~/scripts$ chmod +x random_letters.sh
asivanova@fedora:~/scripts$ ./random_letters.sh
gezundqhqshftluxqkle
asivanova@fedora:~/scripts$ ./random_letters.sh 10
oeyjrknur
asivanova@fedora:~/scripts$

```

Рисунок 2.13: Тестирование random_letters.sh

3 Контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке:

```
while [$1 != "exit"]
```

Ответ: пропущены пробелы после [и перед]. Правильно: `while [ "$1" != "exit" ]`

2. Как объединить (конкатенация) несколько строк в одну? Ответ: можно использовать оператор += или просто записать строки подряд:

```
str="Hello"  
str+=" World"  
# или  
str="${str} World"
```

3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash? Ответ: seq генерирует последовательность чисел. Альтернативы:

```
for i in {1..10}; do echo $i; done  
# или  
for ((i=1; i<=10; i++)); do echo $i; done
```

4. Какой результат даст вычисление выражения `$((10/3))`? Ответ: 3 (целочисленное деление).

5. Укажите кратко основные отличия командной оболочки zsh от bash.

Ответ: zsh имеет автодополнение с подсветкой, поддержку ассоциативных массивов, разделяемые истории между сеансами, более мощные возможности для работы с путями, улучшенную обработку глоббинга.

6. Проверьте, верен ли синтаксис данной конструкции:

```
for ((a=1; a <= LIMIT; a++))
```

Ответ: синтаксис верен.

7. Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки? Ответ:

Преимущества: простота автоматизации команд ОС, встроенные инструменты для работы с файлами и процессами, доступность в UNIX-системах.

Недостатки: медленная работа, ограниченная поддержка чисел с плавающей точкой, неудобный синтаксис для сложных структур, отсутствие объектно-ориентированного программирования.

4 Вывод

В ходе лабораторной работы были разработаны три скрипта:

- `semafor.sh` – реализация упрощённого механизма семафоров с использованием файловой блокировки.
- `my_man.sh` – аналог команды `man` для просмотра справки из `/usr/share/man/man1`.
- `random_letters.sh` – генерация случайной последовательности букв латинского алфавита с использованием `$RANDOM`.

Приобретены навыки межпроцессного взаимодействия через файловые блокировки, работы со сжатыми `man`-страницами (`zcat`), генерации случайных последовательностей, а также изучены особенности синтаксиса `bash`.